FinTech 512 Midterm Spring 2023: Bookshop

Name:_____

NetID: _____

There are four questions, with the point values as shown below. You have 1 hour and 15 minutes. There are 100 points on the exam. Pace yourself accordingly. This exam must be individual work. You may not collaborate with your fellow students. You may use the two sheets of notes you prepared. You may not use any other external resources, e.g., electronic devices. Points will be deducted if you provide more examples than the number listed.

I certify that the work shown on this exam is my own work, and that I have neither given nor received improper assistance of any form in the completion of this work.

Signature: _____

You've been contacted by a local entrepreneur looking for technical help with adapting their IT systems for their music shop, which they are transforming into a bookstore. Here is a recent email the entrepreneur has sent you:

> Hello <your name>,
>
> I really appreciate that you have signed on to help with this project and I trust that you find the terms in the contract acceptable.
>
> As we do not have the budget for full-time technical staff and we focus on customer experience, we would like a single system that can manage books from the time we purchase them through their stocking in the store to their invoicing, delivery, and settlement of the customer's credit card transactions. Since our staff is comfortable with the Flask application a previous consultant built to help us sell new and used music albums, we would like to use the same application while having it modified to work with books. The application interfaces with our Point-Of-Sale system which handles cash, check, and credit-card sales. As we do with the album sales, we would like to review weekly, monthly, and annual sales trends to see what the most popular books and book genres are, and who the most active book buyers and sellers are. We have a loyalty program offering discounts to frequent music buyers, and we are working on the rules for a similar program for book sellers and buyers.
>
> We were impressed with your work on the Chinook database queries and we've heard that you have skills in database design and website development. We are looking forward to your technical advice on how to add books to our existing chinook database.

Question 1: Requirements

a. (12 pts) Identify three functional requirements (FR) and one non-functional requirement (NFR) in the above email. State the requirement as a sentence and indicate its type (FR, NFR.)

1. **FR: a single system that can manage books from the time we purchase them through their stocking in the store to their invoicing, delivery, and settlement of the customer's credit card transactions.**
2. **FR: review weekly, monthly, and annual sales trends**
3. **FR: offer loyalty discounts to book buyers**
4. **NFR: Integrate with existing Flask application**

b. (8 pts) Given the quality attributes listed below, identify the three highest priority quality attributes for the entrepreneur's use case. Explain in a sentence or two why you ranked them in the way that you did.
    a. Quality attributes: performance, efficiency, reliability, security, availability, functionality, modifiability, maintainability, usability, portability, testability, reusability.

1. **Functionality**
2. **Usability**
3. **Modifiability**

NetID: _____

Question 2: Database Design

We have been keeping our records of book sales in a spreadsheet. Here are the first five rows. Can you help us work out what tables we need to add to chinook to support book sales?

Customer	Title	Pages	Price	Authors
Fernanda Ramos	Harry Potter and the Sorcerer's Stone	309	19.95	J.K. Rowling
Mark Philips	The Hobbit	320	9.95	J.R.R. Tolkien
Helena Holy	Exploring Requirements: Quality Before Design	300	39.95	Donald C. Gause, Gerald M. Weinberg
Jimmie Lenz	The Intelligent Investor	640	17.98	Benjamin Graham
Francois Tremblay	Why Zebras Don't Get Ulcers	560	22.99	Robert Sapolsky

[Update the chinook schema (last page) to track the 'spreadsheet' data and link it to the invoice. If you need to modify an existing table, describe the changes to be made to those table(s)]

a. (20 pts) Normalize the data in the given data rows, to third normal form. Show your table (relation) names, field names, and the reformatted data.

Books(id,title,pages,price):
1,Harry Potter and the Sorcerer's Stone,309,19.95
2, The Hobbit,320,9.95
3, Exploring Requirements: Quality Before Design,300,39.95
4, The Intelligent Investor640,17.98
5, Why Zebras Don't Get Ulcers,560,22.99

Authors(id,name)
1,J.K Rowling
2,J.R.R. Tolkien
3,Donald C. Gause
4,Gerald M. Weinberg
5,Benjamin Gtaham
6,Robert Sapolsky

customers(id,fname,lname):
1, Fernanda, Ramos
2,Mark, Philips
3, Helena, Holy
4, Jimmie, Lenz
5,Francois, Tremblay

Book_authors(bookid,authored):
1,1
2,2
3,3
3,4

b. (10 pts) Write the SQL DDL 'CREATE TABLE' statement for one of the relations you defined in part a.

Something like one of the following:

```
CREATE TABLE Books (
   BookId: INTEGER PRIMARY KEY,
   Title: NVARCHAR(160),
   Pages: INTEGER,
   UnitPrice: NUMERIC
)

CREATE TABLE Authors (
   AuthorID: INTEGER PRIMARY KEY,
   FirstName: NVARCHAR(40),
   LastName: NVARCHAR(40)
)

Create table BookAuthors (
   BookID: Integer,
   AuthorId: Integer,
   FOREIGN KEY BookID references books(bookid),
   FOREIGN KEY AuthorId references authors(authorid)
   );
```

Question 3: Testing

The previous contractor developed the code for calculating loyalty program discounts, Python function 'loyalty_discount', **shown on the second to the last page of the exam**. Given that a significant amount of both money and goodwill are at stake in the proper execution of this code, we would like an assessment of how thoroughly the code has been tested.

Here are the test cases the contractor provided:
Invoice has 3 items, invoice total is 110, customer has 2 orders, no onsale items
Invoice has 6 items, invoice total is 150, customer has no orders, no onsale items

a. (10 pts) For the code and the tests given above, what level of coverage (path, condition, statement, none of the above) is achieved?

   None of the above. At least one on sale item on the invoice is required for a set of test cases that provide statement coverage.

b. (10 pts) What test cases need to be added to achieve full (statement, condition, and path) coverage? (You can give just the parameter values, as in the test case examples above)

Statement coverage:

1. **test_statement_contractor1 - < 5 items, no on sale items, 2 customer orders, discount = 0.099**
2. **test_statement_contractor2 - > 5 items, no on sale items, no orders, total > 100, discount = 0.25**
3. **test_statement_onsale1 - > 10 invoice items, > 10 on sale, invoice total > 100, previous customer orders, discount = 0.25**
4. **test_statement_onsale2 - > 10 invoice items, 10 on sale, invoice total > 200, previous customer orders, discount = 0.3**

Branch/condition coverage:
5. test_branch_1_false_invoice_empty – No invoice, no onsale items, discount = 0.099
6. onsale items that aren't on the invoice, discount = 0.25

Path coverage:
7. onsale items not on invoice, total < 100, discount = 0.099 8. onsale items on invoice, total < 100, discount = 0.099

c. (5 pts) What difficulties exist in testing the code as it is currently written?

Question 4: Refactoring

a. (10 pts) We've heard that 'Code Smells' are bad; does the loyalty program code have any? (Name three code smells you identify)

Example Code Smells

Mysterious Name
Divergent Change
Primitive Obsession
Long Function
Global Data
Duplicated Code

b. (15 pts) Can you make suggestions about how to improve the code, given that we intend to apply the same logic to books as well as music? (List three refactorings you could apply to address the code smells identified in part a. Be specific, e.g., if you apply 'Extract Function', give the function name and its parameters.) Write code, noting the name of the refactoring in a comment.

Refactorings

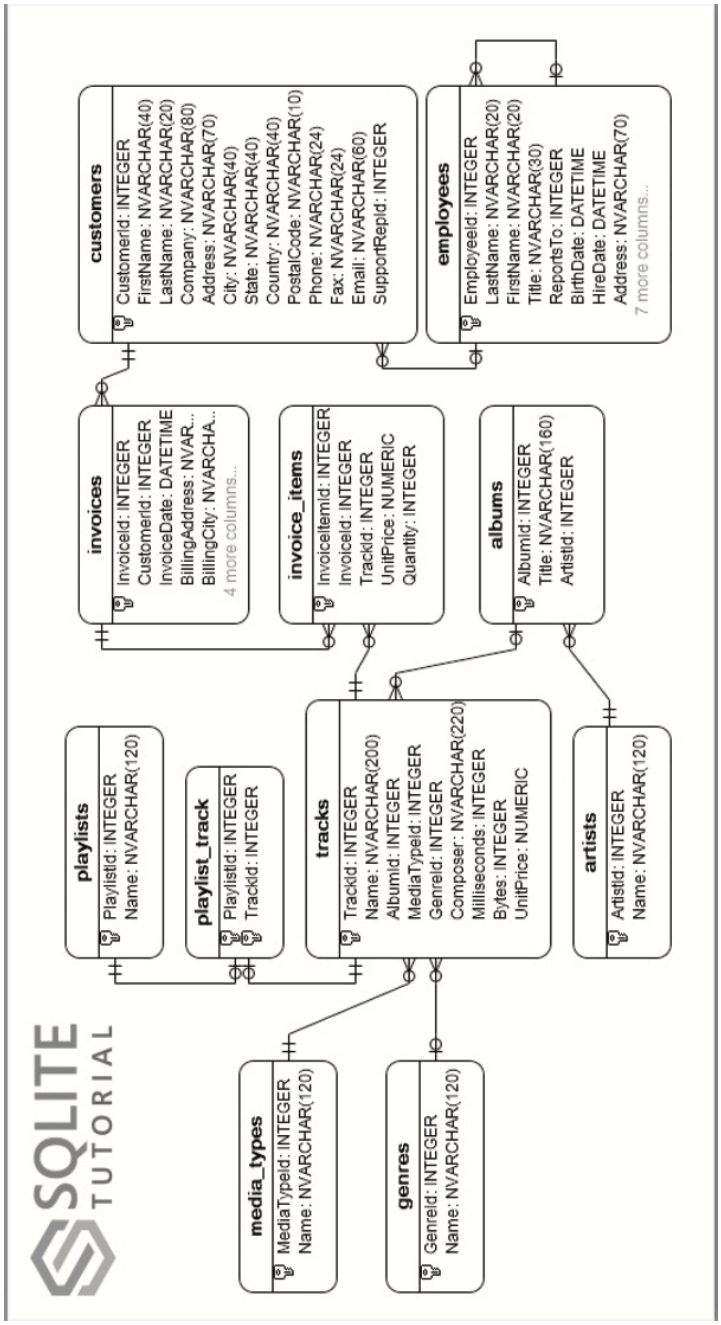
Extract Function (Extract Method) – move database code to its own function(s)
Extract Variable
Extract Class – Order, Invoice
Change Function Declaration – pass in amounts rather than id's
Rename Variable – n, t, s
Replace Primitive with Object – Order, Invoice

```python
def loyalty_discount(invoice_id, customer_id, onsale):
    invoice_items = get_db().execute("SELECT itemid, unitPrice, quantity "
                                     "  FROM INVOICE_ITEMS "
                                     "  WHERE invoiceId = ?",
                                     (invoice_id,)).fetchall()
    order_count = get_db().execute("SELECT count(*) FROM invoices "
                                   "  WHERE customerId = ?",
                                   (customer_id,)).fetchone()

    n = t = s = 0
    for item in invoice_items:
        n += item["quantity"]
        t += item["quantity"] * item["unitPrice"]
        for id in onsale:
            if itemid == id:
                s += 1

    l = max(order_count/10.0,3)
      d = 0.0
    if n > 5 and t > 100.00:
        if s < 10 or t < 200.00:
            d = .25
        else:
            d = .30
        d = min(d,d * (l*.5))
    else:
        d = 0.033 * l

    return d
```


SQLITE
TUTORIAL
customers
CustomerId: INTEGER
FirstName: NVARCHAR(40)
LastName: NVARCHAR(20)
Company: NVARCHAR(80)
Address: NVARCHAR(70)
City: NVARCHAR(40)
State: NVARCHAR(40)
Country: NVARCHAR(40)
PostalCode: NVARCHAR(10)
Phone: NVARCHAR(24)
Fax: NVARCHAR(24)
Email: NVARCHAR(60)
SupportRepId: INTEGER
employees
EmployeeId: INTEGER
LastName: NVARCHAR(20)
FirstName: NVARCHAR(20)
Title: NVARCHAR(30)
ReportsTo: INTEGER
BirthDate: DATETIME
HireDate: DATETIME
Address: NVARCHAR(70)
7 more columns...
invoices
InvoiceId: INTEGER
CustomerId: INTEGER
InvoiceDate: DATETIME
BillingAddress: NVAR...
BillingCity: NVARCHA...
4 more columns...
invoice_items
InvoiceItemId: INTEGER
InvoiceId: INTEGER
TrackId: INTEGER
UnitPrice: NUMERIC
Quantity: INTEGER
albums
AlbumId: INTEGER
Title: NVARCHAR(160)
ArtistId: INTEGER
playlists
PlaylistId: INTEGER
Name: NVARCHAR(120)
playlist_track
PlaylistId: INTEGER
TrackId: INTEGER
tracks
TrackId: INTEGER
Name: NVARCHAR(200)
AlbumId: INTEGER
MediaTypeId: INTEGER
GenreId: INTEGER
Composer: NVARCHAR(220)
Milliseconds: INTEGER
Bytes: INTEGER
UnitPrice: NUMERIC
artists
ArtistId: INTEGER
Name: NVARCHAR(120)
media_types
MediaTypeId: INTEGER
Name: NVARCHAR(120)
genres
GenreId: INTEGER
Name: NVARCHAR(120)